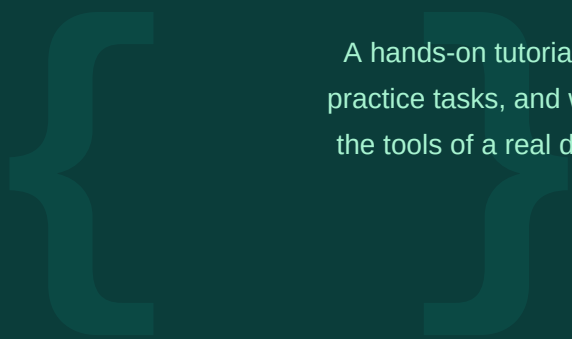




Python for Data Engineers

Files • Error Handling • Logging • Connecting to Systems



A hands-on tutorial with in-depth explanations, annotated code, practice tasks, and worked solutions for each topic — built around the tools of a real data pipeline: MySQL, REST APIs, and Kafka.

Contents

- 00** **How to Use This Tutorial**
Setup, structure, and how tasks/solutions work
- 01** **Working with Files**
Reading, writing, paths, CSV/JSON, binary data
- 02** **Error Handling**
try/except, custom exceptions, clean failure design
- 03** **Logging**
Replacing print(), levels, handlers, formatters, config
- 04** **Connecting to Systems**
MySQL databases, REST APIs, and Kafka basics

How each topic is organized: a detailed explanation of the concept, annotated code examples you can run as-is, one or more comparison/reference tables, then 2–3 practice **Tasks** followed immediately by full **Solutions** with explanations — so you can attempt each task before checking the answer.

00 How to Use This Tutorial

This tutorial covers four building blocks that show up in almost every real-world Python data pipeline: reading and writing **files**, handling failures safely with **error handling**, recording what happened with **logging**, and talking to the outside world through **databases, APIs, and Kafka**. These aren't independent topics in practice — a production ETL script uses all four together: it opens a file, wraps risky operations in try/except, logs every step, and pushes results into a database or a Kafka topic.

Every code example is written to run as-is on Python 3.8+ using only the standard library, except in the "Connecting to Systems" section, which uses `mysql-connector-python`, `requests`, and `kafka-python` — install them with:

TERMINAL

```
pip install mysql-connector-python requests kafka-python
```

Work through each section top to bottom: read the explanation, run the examples yourself, then attempt the tasks before reading the solutions. Typing the code out (rather than copy-pasting) will make the patterns stick much faster.

01 Working with Files

Almost every data engineering script starts and ends with a file: a CSV export from a client system, a JSON payload from an API, a log file, or a staging file before a database load. Python's file handling is built around three ideas: **opening** a file in the right mode, **reading or writing** its contents, and **closing** it reliably — which is why the `with` statement (a context manager) is the standard way to do it.

The with statement and file modes

`open()` returns a file object. Wrapping it in a `with` block guarantees the file is closed automatically, even if an exception is raised inside the block — you never have to remember to call `.close()` yourself.

BASIC FILE READING

```
with open("clients.txt", "r", encoding="utf-8") as f:
    for line in f:
        print(line.strip())
# File is guaranteed to be closed here, even if the loop raised an error
```

Mode	Meaning	Notes
'r'	Read (text)	Default mode. Errors if the file doesn't exist.
'w'	Write (text)	Creates the file, or overwrites it completely.
'a'	Append (text)	Creates the file if missing; writes are added at the end.
'x'	Exclusive create	Fails with an error if the file already exists.
'r+'	Read + write	File must already exist; pointer starts at position 0.
'rb' / 'wb'	Binary read/write	No text decoding — use for images, pickles, etc.

Reading strategies

For small files, read everything at once. For large files (think multi-gigabyte log or export files common in data engineering), iterate line by line so you never hold the whole file in memory.

READING STRATEGIES

```
# Small file: load it all at once
with open("config.json", "r", encoding="utf-8") as f:
    content = f.read()          # single string
    lines = content.splitlines()

# Large file: stream it line by line (memory-efficient)
with open("huge_export.csv", "r", encoding="utf-8") as f:
    for line_number, line in enumerate(f, start=1):
        if line_number % 1_000_000 == 0:
            print(f"Processed {line_number:,} lines")
```

Working with paths: pathlib

The modern, cross-platform way to build and inspect file paths is `pathlib.Path` instead of raw string concatenation. It handles Windows vs. Linux separators for you and reads cleanly.

PATHLIB BASICS

```
from pathlib import Path

data_dir = Path("data") / "incoming"
data_dir.mkdir(parents=True, exist_ok=True)  # create folders if missing

for csv_file in data_dir.glob("*.csv"):
    print(csv_file.name, csv_file.stat().st_size, "bytes")

target = data_dir / "clients_2026.csv"
print(target.exists(), target.suffix, target.parent)
```

Structured formats: CSV and JSON

Raw text handling is rarely enough once data has structure. The `csv` and `json` modules (both in the standard library) handle quoting, delimiters, and type conversion correctly — don't parse CSV or JSON by hand with `.split(',')`.

CSV & JSON

```
import csv
import json

# Reading CSV as dictionaries (column name -> value)
with open("clients.csv", newline="", encoding="utf-8") as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(row["client_name"], row["balance"])

# Writing CSV
with open("summary.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=["client_id", "total"])
    writer.writeheader()
    writer.writerow({"client_id": 101, "total": 4520.50})

# JSON round-trip
with open("settings.json", "r", encoding="utf-8") as f:
    settings = json.load(f)          # JSON text -> Python dict/list

settings["last_run"] = "2026-09-05"
with open("settings.json", "w", encoding="utf-8") as f:
    json.dump(settings, f, indent=2, ensure_ascii=False)
```

Watch out: always pass `newline=""` when opening files for the `csv` module on any platform — otherwise extra blank lines can appear on Windows. Also always set `encoding="utf-8"` explicitly for text you share across machines; relying on the OS default encoding is a classic source of Arabic/UTF-8 mojibake when moving files between Windows and Linux.

Binary files and encodings

Text mode decodes bytes into `str` using an encoding (UTF-8 by default on Linux/Mac). Binary mode (`'rb'` / `'wb'`) hands you raw `bytes` with no decoding at all — use it for images, PDFs, pickled objects, or any non-text payload.

BINARY COPY

```
# Copy a binary file in fixed-size chunks (works for files of any size)
with open("input.bin", "rb") as src, open("output.bin", "wb") as dst:
    while chunk := src.read(65536):  # 64 KB at a time
        dst.write(chunk)
```

Practice Tasks

Task 1 — Log line counter. Write a function `count_lines(path)` that opens a text file and returns the number of non-empty lines, without loading the whole file into memory at once.

Solution:

SOLUTION 1

```
def count_lines(path):
    count = 0
    with open(path, "r", encoding="utf-8") as f:
        for line in f:
            if line.strip():          # skip blank/whitespace-only lines
                count += 1
    return count

print(count_lines("server.log"))
```

Iterating the file object directly (`for line in f`) streams one line at a time from disk instead of loading everything with `.readlines()`, which matters once files get large.

Task 2 — CSV to JSON. Given `clients.csv` with columns `id`, `name`, `balance`, write a script that converts it into `clients.json`: a JSON array of objects, with `balance` converted to a float.

Solution:

SOLUTION 2

```
import csv
import json

def csv_to_json(csv_path, json_path):
    records = []
    with open(csv_path, newline="", encoding="utf-8") as f:
        reader = csv.DictReader(f)
        for row in reader:
            records.append({
                "id": int(row["id"]),
                "name": row["name"],
                "balance": float(row["balance"]),
            })

    with open(json_path, "w", encoding="utf-8") as f:
        json.dump(records, f, indent=2, ensure_ascii=False)

    return len(records)

n = csv_to_json("clients.csv", "clients.json")
print(f"Converted {n} records")
```

Notice the explicit type conversion (`int()`, `float()`) — every value coming out of `csv.DictReader` is a plain string, so numeric fields need to be converted before they're written back out as JSON numbers.

Task 3 — Safe append log. Write `append_event(path, message)` that appends a timestamped line to a log file, creating the file (and its parent folder) if it doesn't exist yet.

Solution:**SOLUTION 3**

```
from pathlib import Path
from datetime import datetime

def append_event(path, message):
    log_path = Path(path)
    log_path.parent.mkdir(parents=True, exist_ok=True)
    timestamp = datetime.now().isoformat(timespec="seconds")
    with open(log_path, "a", encoding="utf-8") as f:
        f.write(f"[{timestamp}] {message}\n")

append_event("logs/pipeline/events.log", "Client sync started")
```

Append mode ('a') never truncates existing content, and `mkdir(parents=True, exist_ok=True)` makes the function safe to call whether or not the `logs/pipeline/` folder already exists — a common pattern in real ingestion scripts.

02 Error Handling

Error handling is about deciding, in advance, what your program should do when something goes wrong — a missing file, a bad row in a CSV, a database that's temporarily down. The goal isn't to prevent every possible error (impossible) but to make failures **predictable**: fail loudly with a clear message, recover when it's safe to, and never leave data half-written or a connection half-open.

try / except / else / finally

Python's error-handling block has four parts. Only `try` and at least one `except` are required.

Clause	Runs when...	Typical use
try	Always — the code you're guarding	The risky operation itself
except	The guarded code raised a matching exception	Handle or log the specific failure
else	The <code>try</code> block succeeded with no exception	Code that should only run on success
finally	Always — exception or not	Cleanup: close files, release locks/connections

TRY / EXCEPT / ELSE / FINALLY

```
def load_balance(row):
    try:
        balance = float(row["balance"])
    except KeyError:
        print("Row is missing a 'balance' column")
        return None
    except ValueError:
        print(f"'{row['balance']}' is not a valid number")
        return None
    else:
        print("Parsed successfully")  # only runs if no exception occurred
        return balance
    finally:
        print("Finished processing row")  # always runs
```

Catch specific exceptions, not everything

A bare `except:` (or `except Exception:` without re-raising) hides bugs — it will silently swallow a typo in your own code just as easily as a real data error. Always catch the narrowest exception type you can, and only widen it deliberately when you have a good reason (e.g. a top-level handler that logs and continues a batch job).

Watch out: `except Exception as e:` `pass` is one of the most common real-world bugs — it makes a program 'never crash' while quietly producing wrong results. If you must catch broadly, at minimum log the exception with its traceback before continuing.

Raising and chaining exceptions

Use `raise` to signal that a function cannot do its job. Inside an `except` block, `raise NewError(...) from original_error` preserves the original traceback as context, which is invaluable when debugging.

EXCEPTION CHAINING

```
def parse_client_id(raw_value):
    try:
        return int(raw_value)
    except ValueError as exc:
        raise ValueError(f"Invalid client id: {raw_value!r}") from exc
```

Custom exceptions

Defining your own exception classes (subclassing `Exception`) lets callers distinguish *your* domain errors from generic Python errors, and lets you attach extra context (like which record failed).

CUSTOM EXCEPTIONS

```
class ClientDataError(Exception):
    """Raised when a client record fails validation."""
    def __init__(self, client_id, reason):
        self.client_id = client_id
        self.reason = reason
        super().__init__(f"Client {client_id}: {reason}")

def validate_client(record):
    if record.get("balance", 0) < 0:
        raise ClientDataError(record.get("id"), "balance cannot be negative")

try:
    validate_client({"id": 42, "balance": -100})
except ClientDataError as exc:
    print(f"Skipping bad record {exc.client_id}: {exc.reason}")
```

Retrying transient failures

Some failures are expected to be temporary — a network blip, a database briefly unreachable. A small retry loop with a delay (and a maximum number of attempts) is a standard, defensive pattern before giving up and raising.

RETRY PATTERN

```
import time

def fetch_with_retry(fetch_fn, max_attempts=3, delay_seconds=2):
    last_error = None
    for attempt in range(1, max_attempts + 1):
        try:
            return fetch_fn()
        except ConnectionError as exc:
            last_error = exc
            print(f"Attempt {attempt} failed: {exc}. Retrying...")
            time.sleep(delay_seconds)
    raise ConnectionError(f"All {max_attempts} attempts failed") from last_error
```

Practice Tasks

Task 1 — Safe division report. Write `safe_divide(a, b)` that returns `a / b`, but returns `None` and prints a clear message on division by zero, without catching any other error type.

Solution:

SOLUTION 1

```
def safe_divide(a, b):
    try:
        return a / b
    except ZeroDivisionError:
        print(f"Cannot divide {a} by zero")
        return None

print(safe_divide(10, 2))    # 5.0
print(safe_divide(10, 0))    # prints message, returns None
```

Catching `ZeroDivisionError` specifically means a real bug elsewhere (like passing a string instead of a number, which raises `TypeError`) will still surface loudly instead of being hidden.

Task 2 — Batch row validator. You have a list of client dicts. Write `validate_batch(records)` that raises `ClientDataError` (from the example above) for any record with a negative balance, but continues checking the rest, and returns a list of the IDs that failed.

Solution:

SOLUTION 2

```
def validate_batch(records):
    failed_ids = []
    for record in records:
        try:
            validate_client(record)
        except ClientDataError as exc:
            print(f"Validation failed: {exc}")
            failed_ids.append(exc.client_id)
    return failed_ids

batch = [
    {"id": 1, "balance": 500},
    {"id": 2, "balance": -20},
    {"id": 3, "balance": -5},
]
print(validate_batch(batch))  # [2, 3]
```

The `try/except` is placed *inside* the loop, per record — this is the key idea for batch processing: one bad row should not stop the other 999 rows from being checked.

Task 3 — Resilient API call. Using the retry pattern above, write a function that simulates calling an unreliable API (it should raise `ConnectionError` the first two times it's called, then succeed) and confirm the retry loop recovers correctly.

Solution:**SOLUTION 3**

```
attempt_counter = {"count": 0}

def unreliable_api_call():
    attempt_counter["count"] += 1
    if attempt_counter["count"] < 3:
        raise ConnectionError("Service temporarily unavailable")
    return {"status": "ok"}

result = fetch_with_retry(unreliable_api_call, max_attempts=5, delay_seconds=1)
print(result)  # {'status': 'ok'} after 2 retries
```

This is a common way to unit-test retry logic without a real flaky network: a small closure or mutable counter simulates “fails twice, then succeeds” so you can verify the loop’s behavior deterministically.

03 Logging

`print()` is fine for quick scripts, but it doesn't scale: you can't turn it off selectively, it has no severity levels, and it doesn't automatically include timestamps, file names, or line numbers. Python's built-in `logging` module solves all of that, and is the standard way any long-running or scheduled script (an ETL job, a cron task, a Kafka consumer) reports what it's doing.

Log levels

Each log call has a severity level. Setting a logger's level filters out anything below it — so you can leave detailed `DEBUG` statements in your code permanently and simply not show them in production.

Level	Value	Use it for
DEBUG	10	Fine-grained detail useful only while developing
INFO	20	Confirmation that things are working as expected
WARNING	30	Something unexpected, but the program can continue
ERROR	40	A specific operation failed
CRITICAL	50	The whole program may be unable to continue

Basic setup

```
BASIC LOGGING SETUP
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s | %(levelname)-8s | %(name)s | %(message)s",
)

logger = logging.getLogger(__name__)

logger.debug("Connecting with timeout=30s") # hidden: below INFO level
logger.info("Sync started for 1,204 clients")
logger.warning("Client 88 has no email on file")
logger.error("Failed to write batch 4 to the database")
```

`logging.getLogger(__name__)` creates a logger named after the current module, which is the convention — it lets you tell at a glance which file a log line came from, and lets you set different levels per module later if needed.

Logging exceptions with a traceback

Inside an `except` block, `logger.exception(...)` automatically attaches the full traceback at ERROR level — far more useful for debugging than logging just the error message.

```
LOGGING EXCEPTIONS

try:
    1 / 0
except ZeroDivisionError:
    logger.exception("Unexpected error while computing ratio")
    # Logs the message AND the full traceback, at ERROR level
```

Handlers: sending logs to files and the console at once

A **handler** decides where log records go. You can attach multiple handlers to one logger — for example, print warnings and above to the console for immediate visibility, while writing everything (including DEBUG) to a rotating log file for later inspection.

```
MULTIPLE HANDLERS

import logging
from logging.handlers import RotatingFileHandler

logger = logging.getLogger("pipeline")
logger.setLevel(logging.DEBUG)

formatter = logging.Formatter(
    "%(asctime)s | %(levelname)-8s | %(message)s"
)

console_handler = logging.StreamHandler()
console_handler.setLevel(logging.WARNING)      # console: warnings and up
console_handler.setFormatter(formatter)

file_handler = RotatingFileHandler(
    "pipeline.log", maxBytes=5_000_000, backupCount=3
)
file_handler.setLevel(logging.DEBUG)          # file: everything
file_handler.setFormatter(formatter)

logger.addHandler(console_handler)
logger.addHandler(file_handler)

logger.debug("Loaded 3 config files")         # file only
logger.warning("Retrying connection")        # file AND console
```

Tip: `RotatingFileHandler` automatically starts a new log file once the current one hits `maxBytes`, keeping `backupCount` old files around — this prevents a long-running service's log file from growing forever and filling the disk.

logging vs. print — quick comparison

	print()	logging
Severity levels	No	Yes (DEBUG→CRITICAL)
Can be disabled selectively	No	Yes, per logger/handler
Timestamps / source info	Manual	Automatic via formatter
Multiple destinations	No	Yes (console, file, network, ...)
Safe to leave in production code	Usually not	Yes, by design

Practice Tasks

Task 1 — Module logger. Set up basic logging with `logging.basicConfig` at `INFO` level, then write a function `process_file(path)` that logs an `INFO` message when it starts, a `WARNING` if the file is empty, and returns the line count.

Solution:

SOLUTION 1

```
import logging

logging.basicConfig(level=logging.INFO,
                    format="%(asctime)s | %(levelname)-8s | %(message)s")
logger = logging.getLogger(__name__)

def process_file(path):
    logger.info(f"Processing {path}")
    with open(path, "r", encoding="utf-8") as f:
        lines = [line for line in f if line.strip()]
    if not lines:
        logger.warning(f"{path} is empty")
    return len(lines)

process_file("clients.txt")
```

Task 2 — Log + re-raise. Write a function `load_config(path)` that reads a JSON config file. If the file is missing or the JSON is invalid, log the full exception with `logger.exception()` and then re-raise it so the caller still sees the failure.

Solution:

SOLUTION 2

```
import json
import logging

logger = logging.getLogger(__name__)

def load_config(path):
    try:
        with open(path, "r", encoding="utf-8") as f:
            return json.load(f)
    except (FileNotFoundError, json.JSONDecodeError):
        logger.exception(f"Failed to load config from {path}")
        raise
```

Logging *and* re-raising is a deliberate combination: the log gives you a permanent record for later debugging, while re-raising still lets the immediate caller (or the program) stop or handle the failure right away.

Task 3 — Dual handler setup. Configure a logger named "etl" so that DEBUG and above goes to `etl_debug.log`, while only ERROR and above is also printed to the console.

Solution:**SOLUTION 3**

```
import logging

logger = logging.getLogger("etl")
logger.setLevel(logging.DEBUG)

fmt = logging.Formatter("%(asctime)s | %(levelname)-8s | %(message)s")

file_handler = logging.FileHandler("etl_debug.log")
file_handler.setLevel(logging.DEBUG)
file_handler.setFormatter(fmt)

console_handler = logging.StreamHandler()
console_handler.setLevel(logging.ERROR)
console_handler.setFormatter(fmt)

logger.addHandler(file_handler)
logger.addHandler(console_handler)

logger.debug("Row 4021 skipped: empty email") # file only
logger.error("Database connection lost")      # file AND console
```


04 Connecting to Systems

This section ties the previous three together: connecting to a **MySQL database**, calling a **REST API**, and producing/consuming messages with **Kafka** — three connection types you'll use constantly in a data engineering pipeline. In every case, the same principles from before apply: use context managers to guarantee cleanup, wrap network/DB calls in specific try/except blocks, and log what happened.

4.1 Databases (MySQL)

The `mysql-connector-python` package provides a DB-API 2.0 compatible connection to MySQL. The two rules that matter most: **always use parameterized queries** (never build SQL with string formatting/f-strings — that's how SQL injection happens), and always close connections and cursors, ideally via context managers.

```
MYSQL: SAFE QUERYING
import mysql.connector
from mysql.connector import Error
import logging

logger = logging.getLogger(__name__)

def get_active_clients(host, user, password, database):
    try:
        conn = mysql.connector.connect(
            host=host, user=user, password=password, database=database
        )
        with conn.cursor(dictionary=True) as cursor:
            # Parameterized query -- the %s placeholder is filled in safely,
            # never build this string with f"...{status}..."
            cursor.execute(
                "SELECT id, name, balance FROM clients WHERE status = %s",
                ("active",)
            )
            return cursor.fetchall()
    except Error as exc:
        logger.exception("MySQL query failed")
        raise
    finally:
        if "conn" in locals() and conn.is_connected():
            conn.close()

clients = get_active_clients("localhost", "app_user", "secret", "client_management")
for c in clients:
    print(c["name"], c["balance"])
```

For inserting many rows efficiently (a common ETL step), use `executemany()` instead of looping over single inserts — it batches the round-trips to the server.

BATCH INSERTS

```
def insert_transactions(conn, transactions):
    """transactions: list of (client_id, amount, occurred_at) tuples"""
    sql = """
        INSERT INTO transactions (client_id, amount, occurred_at)
        VALUES (%s, %s, %s)
    """
    with conn.cursor() as cursor:
        cursor.executemany(sql, transactions)
    conn.commit()          # MySQL connections default to non-autocommit
```

Watch out: after any INSERT/UPDATE/DELETE with `mysql-connector-python`, you must call `conn.commit()` or the changes are rolled back when the connection closes. If something fails mid-batch, call `conn.rollback()` in the except block to avoid a half-applied batch.

4.2 REST APIs

The `requests` library is the standard way to call HTTP APIs. Three things matter for reliable API calls: setting a **timeout** (a hung request without one can block your script forever), checking the response status, and handling network errors separately from bad response codes.

CALLING A REST API

```
import requests
from requests.exceptions import RequestException
import logging

logger = logging.getLogger(__name__)

def fetch_exchange_rate(base_currency, target_currency):
    url = f"https://api.example.com/rates/{base_currency}"
    try:
        response = requests.get(
            url,
            params={"to": target_currency},
            headers={"Authorization": "Bearer YOUR_TOKEN"},
            timeout=10,          # never call an API without a timeout
        )
        response.raise_for_status()  # raises HTTPError for 4xx/5xx status codes
    except RequestException as exc:
        logger.exception(f"Failed to fetch rate {base_currency}->{target_currency}")
        raise
    return response.json()

rate = fetch_exchange_rate("USD", "EGP")
print(rate)
```

Combine this with the retry pattern from the Error Handling section for APIs that occasionally rate-limit or time out:

RETRYING API CALLS

```
import time
import requests

def get_with_retry(url, max_attempts=3, backoff=2, **kwargs):
    for attempt in range(1, max_attempts + 1):
        try:
            response = requests.get(url, timeout=10, **kwargs)
            if response.status_code == 429:  # too many requests
                wait = backoff ** attempt
                logger.warning(f"Rate limited, waiting {wait}s")
                time.sleep(wait)
                continue
            response.raise_for_status()
            return response
        except requests.exceptions.ConnectionError:
            logger.warning(f"Connection error on attempt {attempt}")
            time.sleep(backoff ** attempt)
    raise RuntimeError(f"Failed to GET {url} after {max_attempts} attempts")
```

4.3 Kafka Basics

Kafka decouples systems by moving data through **topics**: **producers** write messages to a topic, and **consumers** read them, independently and at their own pace. Since you're already working with HDFS-to-Kafka streaming, the pattern below matches that same producer/consumer shape, using [kafka-python](#).

KAFKA PRODUCER

```
import json
from kafka import KafkaProducer
import logging

logger = logging.getLogger(__name__)

producer = KafkaProducer(
    bootstrap_servers=["localhost:9092"],
    value_serializer=lambda v: json.dumps(v).encode("utf-8"),
    acks="all",          # wait for full replica acknowledgement
    retries=3,
)

def publish_client_event(client_id, event_type):
    message = {"client_id": client_id, "event": event_type}
    try:
        future = producer.send("client-events", value=message)
        future.get(timeout=10) # block until the broker confirms, or raise
        logger.info(f"Published {event_type} for client {client_id}")
    except Exception:
        logger.exception("Failed to publish message to Kafka")
        raise

publish_client_event(101, "balance_updated")
producer.flush()
producer.close()
```

KAFKA CONSUMER

```

from kafka import KafkaConsumer
import json
import logging

logger = logging.getLogger(__name__)

consumer = KafkaConsumer(
    "client-events",
    bootstrap_servers=["localhost:9092"],
    value_deserializer=lambda v: json.loads(v.decode("utf-8")),
    group_id="client-events-processor", # consumers in the same group share the load
    auto_offset_reset="earliest",      # read from the start if no offset is stored
    enable_auto_commit=True,
)

try:
    for record in consumer:
        event = record.value
        logger.info(f"Received event: {event}")
        # process event here (e.g. write to MySQL, update a dashboard, ...)
except KeyboardInterrupt:
    logger.info("Shutting down consumer")
finally:
    consumer.close()

```

Concept	What it means
Topic	A named stream of messages (e.g. "client-events")
Producer	Writes messages to a topic
Consumer	Reads messages from a topic
Consumer group	Consumers sharing a group_id split the topic's partitions between them
Offset	A message's position in a partition; tracks how far a consumer has read
acks='all'	Producer waits for all in-sync replicas to confirm — safest, slowest option

Tip: for a real ingestion pipeline, wrap the consumer's per-message processing in its own try/except so one malformed message doesn't crash the whole consumer loop — log the bad message and continue to the next one, the same pattern used for batch row validation earlier.

Practice Tasks

Task 1 — Parameterized lookup. Write `get_client_by_id(conn, client_id)` using `mysql-connector-python` that safely looks up one client by ID (parameterized, not string-formatted) and returns `None` if no row is found.

Solution:

SOLUTION 1

```
def get_client_by_id(conn, client_id):
    with conn.cursor(dictionary=True) as cursor:
        cursor.execute(
            "SELECT id, name, balance FROM clients WHERE id = %s",
            (client_id,)
        )
        row = cursor.fetchone()
        return row    # None if no matching row

client = get_client_by_id(conn, 101)
print(client or "Not found")
```

The tuple `(client_id,)` — note the trailing comma — is required even for a single parameter; this is what keeps the value safely separated from the SQL text.

Task 2 — Resilient API fetch with logging. Combine the `get_with_retry` function with logging so that every failed attempt is logged as a `WARNING` and the final give-up is logged as `ERROR` before the exception propagates.

Solution:

SOLUTION 2

```

import logging
import time
import requests

logger = logging.getLogger(__name__)

def get_with_logging(url, max_attempts=3, backoff=2, **kwargs):
    for attempt in range(1, max_attempts + 1):
        try:
            response = requests.get(url, timeout=10, **kwargs)
            response.raise_for_status()
            return response
        except requests.exceptions.RequestException as exc:
            logger.warning(f"Attempt {attempt}/{max_attempts} failed: {exc}")
            if attempt == max_attempts:
                logger.error(f"Giving up on {url} after {max_attempts} attempts")
                raise
            time.sleep(backoff ** attempt)

```

Task 3 — Kafka producer with error handling. Write `safe_publish(producer, topic, message)` that sends a message, waits for confirmation with a 5-second timeout, logs success at INFO and failure at ERROR, and returns `True/False` instead of raising.

Solution:**SOLUTION 3**

```

import json
import logging

logger = logging.getLogger(__name__)

def safe_publish(producer, topic, message):
    try:
        future = producer.send(topic, value=message)
        future.get(timeout=5)
        logger.info(f"Published to {topic}: {message}")
        return True
    except Exception:
        logger.exception(f"Failed to publish to {topic}")
        return False

ok = safe_publish(producer, "client-events",
                  {"client_id": 205, "event": "profile_updated"})

if not ok:
    # e.g. write the message to a local dead-letter file for later retry
    pass

```

Returning a boolean instead of letting the exception propagate is a deliberate choice here: it lets the caller decide what “Kafka is down” should mean for the rest of the pipeline — for example, falling back to writing failed events to a local dead-letter file instead of stopping the whole batch.

Where to go next: try chaining all four topics together — read client records from a CSV file (Section 1), validate each one and catch bad rows (Section 2), log every step to a rotating file (Section 3), then insert valid records into MySQL and publish a Kafka event per client processed (Section 4). That end-to-end script is close to what a real ingestion job looks like.